IMPLEMENTASI BRIGHTNESS ADJUSTMENT PADA CITRA DIGITAL MENGGUNAKAN PYTHON

Disusun untuk memenuhi Tugas Mata Kuliah Pengolahan Citra Digital

Dosen Pengampu: Bapak Andi Aljabar, S.Kom.,M.T.I.



Disusun Oleh:

Ade Ayu Wandira (24260007)

UNIVERSITAS NAHDLATUL ULAMA INDONESIA

FAKULTAS TEKNIK DAN ILMU KOMPUTER

2026

A. Latar Belakang

Citra digital adalah representasi gambar dalam bentuk kumpulan titik terkecil yang disebut piksel. Piksel-piksel tersebut tersusun dalam baris dan kolom, dan setiap piksel menyimpan nilai intensitas yang menentukan warna serta tingkat terang atau gelapnya titik itu. Pada citra warna 8-bit, setiap kanal warna memiliki nilai intensitas dari 0 (paling gelap) sampai 255 (paling terang). Dengan demikian, bagi komputer sebuah gambar tidak lain adalah susunan angka yang dapat dibaca dan dihitung.

Pengolahan citra digital diperlukan karena gambar yang diperoleh tidak selalu berada pada kondisi yang ideal. Pencahayaan saat pengambilan gambar dapat membuat hasil terlalu gelap atau terlalu terang, sehingga detail objek sulit diamati. Melalui pengolahan citra, nilai piksel dapat dimodifikasi untuk memperbaiki tampilan, memperjelas informasi visual, dan menyiapkan gambar untuk tahap analisis berikutnya.

Brightness Adjustment adalah salah satu metode pengolahan citra yang paling sederhana untuk mengubah kecerahan gambar. Metode ini menambahkan atau mengurangi sebuah nilai tetap pada intensitas setiap piksel. Nilai positif membuat gambar lebih terang, sedangkan nilai negatif membuatnya lebih gelap. Laporan ini membahas penerapan metode tersebut menggunakan bahasa pemrograman Python dengan library OpenCV, NumPy, dan Matplotlib pada sebuah gambar buket bunga, dengan perubahan kecerahan +50 dan -50. Hasilnya dibandingkan secara visual dan secara numerik melalui nilai minimum, maksimum, serta rata-rata intensitas piksel.

B. Pembahasan Program

Program ditulis pada Jupyter Notebook dengan nama berkas `Brightness_Adjustment.ipynb` dan terdiri dari sembilan tahap kode. Setiap tahap di bawah ini dibahas berurutan sesuai cell pada notebook, mulai dari kode program, penjelasan setiap barisnya, hasil eksekusi, hingga makna hasil tersebut.

1. Import Library

Tahap pertama memuat semua library yang dibutuhkan program. Library adalah kumpulan fungsi siap pakai, sehingga pembacaan gambar, perhitungan piksel, dan penampilan hasil tidak perlu ditulis dari awal.

Kode program:

```
# ==============================
# 1. IMPORT LIBRARY
# ==============================

import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
```

✓ 0.0s

Penjelasan kode:

- Tiga baris komentar di bagian awal (`# ====`, judul tahap, dan `# ====`) hanya berfungsi sebagai penanda tahap. Python mengabaikan semua teks setelah tanda `#`, sehingga baris ini tidak memengaruhi jalannya program.
- `import cv2` mengimpor OpenCV, library pengolahan citra. Pada program ini `cv2` dipakai untuk membaca gambar (`cv2.imread`), mengubah urutan kanal warna (`cv2.cvtColor`), dan menyimpan gambar (`cv2.imwrite`).
- `import numpy as np` mengimpor NumPy dengan alias `np`. Gambar yang dibaca OpenCV disimpan sebagai array NumPy, sehingga NumPy digunakan untuk menghitung seluruh piksel sekaligus, misalnya melalui `astype`, `clip`, `min`, `max`, dan `mean`.
- `import matplotlib.pyplot as plt` mengimpor modul `pyplot` dari Matplotlib dengan alias `plt`. Modul ini dipakai untuk menampilkan gambar dan membuat visualisasi perbandingan (`figure`, `imshow`, `subplot`, `show`).
- `import os` mengimpor modul `os` bawaan Python. Pada program ini fungsi `os.path.getsize` dipakai untuk mengetahui ukuran berkas gambar.
- Kata kunci `import` memuat sebuah library agar fungsinya dapat dipakai, sedangkan `as` memberi nama alias yang lebih singkat (`np` dan `plt`) agar penulisan kode berikutnya lebih ringkas.

Hasil eksekusi:

Cell ini tidak menampilkan output apa pun. Tidak adanya pesan kesalahan menunjukkan bahwa keempat library berhasil dimuat dan siap digunakan pada tahap berikutnya. Hasil tersebut wajar karena baris `import` hanya memuat library dan tidak memiliki perintah `print`. Keberhasilan tahap ini terbukti dari tahap-tahap selanjutnya yang dapat berjalan tanpa kesalahan.

2. Membaca Gambar

Pada tahap ini gambar asli dibaca dari berkas dan dimuat ke memori sebagai data piksel yang siap diolah.

Kode program:

```
# ==============================
# 2. MEMBACA GAMBAR ASLI
# ==============================

# Menentukan lokasi file gambar
lokasi_gambar = "foto asli.jpg"

# Membaca gambar menggunakan OpenCV
gambar_asli = cv2.imread(lokasi_gambar)

# Memeriksa apakah gambar berhasil dibaca
if gambar_asli is None:
    print("Gambar gagal dibaca. Periksa nama dan lokasi file.")
else:
    print("Gambar berhasil dibaca.")
```

✓ 0.0s

Penjelasan kode:

- `lokasi_gambar = "foto asli.jpg"` membuat variabel `lokasi_gambar` yang berisi teks (string) nama berkas gambar. Karena hanya nama berkas yang ditulis, berkas tersebut harus berada di folder yang sama dengan notebook.
- `gambar_asli = cv2.imread(lokasi_gambar)` memanggil fungsi `cv2.imread` untuk membaca berkas gambar. Parameternya adalah lokasi berkas. Jika berhasil, fungsi mengembalikan array NumPy berisi nilai piksel (tinggi x lebar x 3 kanal, urutan BGR, tipe `uint8`) yang disimpan di variabel `gambar_asli`. Jika gagal, fungsi tidak menimbulkan error, tetapi mengembalikan `None`.
- `if gambar_asli is None:` memeriksa apakah hasil pembacaan bernilai `None`, yaitu tanda bahwa berkas tidak ditemukan atau tidak dapat dibaca.
- `print("Gambar gagal dibaca. Periksa nama dan lokasi file.")` hanya dijalankan jika kondisi `if` terpenuhi, dan menampilkan pesan agar pengguna memeriksa nama serta lokasi berkas.
- `else:` menjalankan blok setelahnya apabila kondisi `if` tidak terpenuhi, artinya gambar berhasil dibaca.
- `print("Gambar berhasil dibaca.")` menampilkan pesan bahwa gambar telah dimuat dan siap digunakan.

Hasil eksekusi:

```
Gambar berhasil dibaca.
```

Pesan "Gambar berhasil dibaca." menunjukkan bahwa `gambar_asli` tidak bernilai `None`, sehingga berkas `foto asli.jpg` ditemukan dan datanya telah dimuat. Pesan gagal tidak tampil karena blok `if` tidak dijalankan. Pemeriksaan ini penting agar tahap berikutnya tidak berjalan pada data yang kosong.

3. Menampilkan Gambar Asli

Gambar asli ditampilkan terlebih dahulu sebagai acuan sebelum kecerahannya diubah.

Kode program:

```
# ==============================
# 3. MENAMPILKAN GAMBAR ASLI
# ==============================

# Mengubah format warna BGR menjadi RGB
gambar_rgb = cv2.cvtColor(gambar_asli, cv2.COLOR_BGR2RGB)

# Mengatur ukuran visualisasi gambar
plt.figure(figsize=(5, 6))

# Menampilkan gambar asli
plt.imshow(gambar_rgb)
plt.title("Gambar Asli")
plt.axis("off")

# Merapikan tata letak visualisasi
plt.tight_layout()
plt.show()
```

0.2s

Penjelasan kode:

- `gambar_rgb = cv2.cvtColor(gambar_asli, cv2.COLOR_BGR2RGB)` mengubah urutan kanal warna dari BGR menjadi RGB. OpenCV menyimpan warna dengan urutan biru-hijau-merah, sedangkan Matplotlib mengharapkan merah-hijau-biru. Tanpa konversi ini warna gambar akan tampil keliru, misalnya merah muda terlihat kebiruan. Hasilnya disimpan di variabel `gambar_rgb`.
- `plt.figure(figsize=(5, 6))` membuat area gambar (figure) berukuran lebar 5 inci dan tinggi 6 inci.
- `plt.imshow(gambar_rgb)` menggambar data piksel `gambar_rgb` pada area tersebut.
- `plt.title("Gambar Asli")` memberi judul "Gambar Asli" di atas gambar.
- `plt.axis("off")` menyembunyikan sumbu beserta angka koordinatnya agar tampilan bersih.

- `plt.tight_layout()` mengatur jarak antarelemen supaya judul dan gambar tersusun rapi dan tidak terpotong.
- `plt.show()` menampilkan figure pada output notebook.

Hasil eksekusi:



Gambar 1. Hasil eksekusi tahap Menampilkan Gambar Asli

Cell ini tidak mencetak teks, tetapi menghasilkan satu visualisasi berukuran 500 x 600 piksel dengan satu sumbu (`<Figure size 500x600 with 1 Axes>`), seperti terlihat pada Gambar 1. Gambar menampilkan buket bunga merah muda dengan latar gelap dan judul "Gambar Asli", dan warna tampil wajar sehingga konversi BGR ke RGB berjalan benar. Keadaan awal inilah yang menjadi pembanding bagi hasil Brightness Adjustment.

4. Informasi Gambar Asli

Tahap ini mengumpulkan karakteristik gambar asli, yaitu dimensi, jumlah piksel, tipe data, ukuran berkas, dan statistik intensitas, sebagai data pembanding sebelum gambar diubah.

Kode program:

```
# ==============================
# 4. INFORMASI GAMBAR ASLI
# ==============================

# Mengambil dimensi dan jumlah kanal gambar
tinggi, lebar, kanal = gambar_asli.shape

# Mengambil tipe data piksel
tipe_data = gambar_asli.dtype

# Menghitung jumlah seluruh piksel
jumlah_piksel = tinggi * lebar

# Menghitung ukuran file dalam KB
ukuran_file = os.path.getsize(lokasi_gambar) / 1024

# Menghitung nilai intensitas piksel
nilai_minimum = np.min(gambar_asli)
nilai_maksimum = np.max(gambar_asli)
nilai_rata_rata = np.mean(gambar_asli)

# Menyusun informasi gambar
informasi_gambar = [
    ["Nama file", lokasi_gambar],
    ["Sumber gambar", "Pinterest"],
    ["Format file", lokasi_gambar.split(".")[-1].upper()],
    ["Lebar gambar", f"{lebar} piksel"],
    ["Tinggi gambar", f"{tinggi} piksel"],
    ["Resolusi spasial", f"{lebar} x {tinggi} piksel"],
    ["Jumlah piksel", f"{jumlah_piksel:,} piksel"],
    ["Jumlah kanal warna", str(kanal)],
    ["Urutan kanal", "BGR (OpenCV)"],
    ["Tipe data piksel", str(tipe_data)],
    ["Ukuran file", f"{ukuran_file:.2f} KB"],
    ["Intensitas minimum", str(nilai_minimum)],
    ["Intensitas maksimum", str(nilai_maksimum)],
    ["Rata-rata intensitas", f"{nilai_rata_rata:.2f}"],
]

# Menampilkan informasi dalam bentuk teks dan tabel sederhana
print("INFORMASI GAMBAR ASLI")
print("=" * 48)
print(f"{'Informasi':<25} | {'Nilai'}")
print("-" * 48)

for informasi, nilai in informasi_gambar:
    print(f"{informasi:<25} | {nilai}")

print("=" * 48)
```

✓ 0.0s

Penjelasan kode:

- Komentar `# Mengambil dimensi dan jumlah kanal gambar`, `# Mengambil tipe data piksel`, dan komentar sejenis hanya penanda dan tidak dijalankan.
- `tinggi, lebar, kanal = gambar_asli.shape` membaca atribut `shape`, yaitu tuple berisi tinggi, lebar, dan jumlah kanal array gambar. Ketiga nilainya langsung dipecah ke variabel `tinggi`, `lebar`, dan `kanal`.
- `tipe_data = gambar_asli.dtype` mengambil tipe data setiap nilai piksel. Pada gambar 8-bit tipenya `uint8`, yaitu bilangan bulat tanpa tanda dengan rentang 0-255.
- `jumlah_piksel = tinggi * lebar` mengalikan tinggi dan lebar untuk memperoleh jumlah piksel. Jumlah kanal tidak ikut dikalikan karena yang dihitung adalah piksel, bukan seluruh nilai kanal.
- `ukuran_file = os.path.getsize(lokasi_gambar) / 1024` mengambil ukuran berkas dalam byte, lalu membaginya dengan 1024 untuk mendapat ukuran dalam kilobyte (KB).
- `nilai_minimum = np.min(gambar_asli)` mencari nilai intensitas terkecil dari seluruh nilai kanal pada gambar.
- `nilai_maksimum = np.max(gambar_asli)` mencari nilai intensitas terbesar dari seluruh nilai kanal pada gambar.
- `nilai_rata_rata = np.mean(gambar_asli)` menghitung rata-rata seluruh nilai kanal pada gambar.
- `informasi_gambar = [ ... ]` membuat daftar (list) yang setiap elemennya berupa pasangan [nama informasi, nilai]. Beberapa nilai ditulis langsung sebagai teks, yaitu "Sumber gambar" (`"Pinterest"`) dan "Urutan kanal" (`"BGR (OpenCV)"`), sedangkan nilai lainnya dihitung dari variabel sebelumnya.
- `lokasi_gambar.split(".")[-1].upper()` memecah teks `foto asli.jpg` pada tanda titik menjadi `["foto asli", "jpg"]`, mengambil elemen terakhir dengan indeks `[-1]`, lalu mengubahnya menjadi huruf kapital sehingga menghasilkan `JPG`.
- Awalan `f` pada teks seperti `f"{lebar} piksel"` menandakan f-string, yaitu teks yang dapat menyisipkan nilai variabel di dalam tanda `{}`. Format `{jumlah_piksel:,}` menambahkan pemisah ribuan, dan `{ukuran_file:.2f}` membulatkan angka menjadi dua desimal. Fungsi `str()` mengubah angka menjadi teks agar seluruh nilai pada daftar bertipe sama.
- `print("INFORMASI GAMBAR ASLI")` mencetak judul tabel, sedangkan `print("=" * 48)` dan `print("-" * 48)` mencetak karakter = dan - sebanyak 48 kali sebagai garis pemisah.
- `print(f"{'Informasi':<25} | {'Nilai'}")` mencetak baris kepala tabel. Format `:<25` meratakan teks ke kiri pada lebar 25 karakter sehingga kolom sejajar.

- `for informasi, nilai in informasi_gambar:` mengulang daftar, mengambil satu pasangan pada setiap putaran. `print(f"{informasi:<25} | {nilai}")` mencetak pasangan tersebut sebagai satu baris tabel. Perulangan berjalan sebanyak 14 kali sesuai jumlah elemen daftar.
- `print("=" * 48)` terakhir mencetak garis penutup tabel.

Hasil eksekusi:

```
INFORMASI GAMBAR ASLI
================================================
Informasi                | Nilai
------------------------------------------------
Nama file                | foto asli.jpg
Sumber gambar            | Pinterest
Format file              | JPG
Lebar gambar             | 736 piksel
Tinggi gambar            | 981 piksel
Resolusi spasial         | 736 x 981 piksel
Jumlah piksel            | 722,016 piksel
Jumlah kanal warna       | 3
Urutan kanal             | BGR (OpenCV)
Tipe data piksel         | uint8
Ukuran file              | 79.37 KB
Intensitas minimum       | 0
Intensitas maksimum      | 255
Rata-rata intensitas     | 63.58
================================================
```

Gambar berukuran 736 x 981 piksel sehingga memiliki 736 x 981 = 722.016 piksel. Gambar memiliki 3 kanal warna (BGR) dengan tipe data `uint8`, dan ukuran berkasnya 79,37 KB.

Nilai intensitas minimum 0 dan maksimum 255 menunjukkan bahwa gambar sudah memuat nilai paling gelap dan paling terang yang mungkin. Rata-rata intensitas sebesar 63,58 berada jauh di bawah titik tengah rentang (127,5), sehingga secara keseluruhan gambar tergolong gelap. Hal ini sesuai dengan tampilan gambar yang berlatar gelap. Ketiga angka ini nantinya dibandingkan dengan hasil Brightness Adjustment.

5. Menentukan Nilai Kecerahan

Tahap ini menetapkan besar perubahan kecerahan yang akan diterapkan pada gambar.

Kode program:

```
# ==============================
# 5. MENENTUKAN NILAI KECERAHAN
# ==============================

# Nilai positif untuk mencerahkan gambar
nilai_terang = 50

# Nilai negatif untuk menggelapkan gambar
nilai_gelap = -50

# Menampilkan nilai perubahan kecerahan
print("Nilai perubahan kecerahan")
print(f"Gambar lebih terang : {nilai_terang}")
print(f"Gambar lebih gelap   : {nilai_gelap}")
```

✓ 0.0s

Penjelasan kode:

- `nilai_terang = 50` membuat variabel yang berisi bilangan positif 50, yaitu nilai yang akan ditambahkan ke setiap piksel agar gambar lebih terang.
- `nilai_gelap = -50` membuat variabel yang berisi bilangan negatif -50. Menambahkan nilai negatif sama dengan mengurangi 50 dari setiap piksel sehingga gambar lebih gelap.
- `print("Nilai perubahan kecerahan")` mencetak judul keterangan.
- `print(f"Gambar lebih terang : {nilai_terang}")` dan `print(f"Gambar lebih gelap : {nilai_gelap}")` mencetak nilai kedua variabel. Awalan `f` dan tanda `{}` menyisipkan isi variabel ke dalam teks. Spasi tambahan pada teks kedua dipakai agar tanda titik dua sejajar.

Hasil eksekusi:

```
Nilai perubahan kecerahan
Gambar lebih terang : 50
Gambar lebih gelap   : -50
```

Output menegaskan bahwa gambar terang dibuat dengan perubahan +50 dan gambar gelap dengan -50. Besar perubahan yang sama dengan arah berlawanan memudahkan perbandingan hasil keduanya. Nilai ini disimpan dalam variabel sehingga dapat diganti dengan mudah tanpa mengubah kode pada tahap pengolahan.

6. Melakukan Brightness Adjustment

Inilah tahap inti program, yaitu mengubah intensitas seluruh piksel dengan menambahkan nilai kecerahan. Secara matematis, setiap piksel diubah dengan rumus I'(x, y) = batasi(I(x, y) + b), dengan I adalah intensitas piksel awal, b adalah nilai kecerahan (+50 atau -50), dan fungsi batasi menjaga hasilnya tetap pada rentang 0-255.

Kode program:

```
# ==============================
# 6. MELAKUKAN BRIGHTNESS ADJUSTMENT
# ==============================

# Membuat gambar lebih terang
gambar_terang = np.clip(
    gambar_asli.astype(np.int16) + nilai_terang, 0, 255
).astype(np.uint8)

# Membuat gambar lebih gelap
gambar_gelap = np.clip(
    gambar_asli.astype(np.int16) + nilai_gelap, 0, 255
).astype(np.uint8)

print("Proses Brightness Adjustment berhasil.")
```

✓ 0.0s

Penjelasan kode:

- `gambar_terang = np.clip(...)` membuat variabel `gambar_terang` untuk menyimpan hasil gambar yang lebih terang. Pernyataan ditulis dalam beberapa baris di dalam tanda kurung agar mudah dibaca.
- `gambar_asli.astype(np.int16)` mengubah tipe data array dari `uint8` menjadi `int16` secara sementara. Tipe `uint8` hanya menampung 0-255, sehingga hasil penjumlahan seperti 250 + 50 akan "berputar" kembali ke angka kecil (overflow), dan 20 - 50 akan berputar menjadi angka besar. Tipe `int16` mampu menampung nilai negatif dan nilai di atas 255 sehingga hasil perhitungan tetap benar.
- `+ nilai_terang` menambahkan 50 ke setiap nilai pada array, yaitu ke semua piksel dan semua kanal warna sekaligus tanpa perulangan.
- `np.clip(..., 0, 255)` membatasi hasil perhitungan: nilai di bawah 0 diubah menjadi 0, nilai di atas 255 diubah menjadi 255, dan nilai di antaranya tidak berubah. Pembatasan ini disebut clipping dan menjaga intensitas tetap pada rentang yang sah.
- `.astype(np.uint8)` mengembalikan tipe data hasil ke `uint8` agar sesuai dengan format gambar 8-bit yang dapat ditampilkan dan disimpan.
- `gambar_gelap = np.clip(gambar_asli.astype(np.int16) + nilai_gelap, 0, 255).astype(np.uint8)` bekerja dengan langkah yang sama, tetapi menggunakan `nilai_gelap` (-50). Hasilnya, setiap piksel dikurangi 50 dan nilai yang kurang dari 0 dipotong menjadi 0.
- `print("Proses Brightness Adjustment berhasil.")` menampilkan pesan bahwa kedua gambar hasil telah dibuat.

Hasil eksekusi:

```
...  Proses Brightness Adjustment berhasil.
```

Output hanya berupa pesan keberhasilan karena hasil pengolahan disimpan dalam variabel `gambar_terang` dan `gambar_gelap`, bukan dicetak. Perubahan intensitas yang sebenarnya dapat dilihat pada tahap perbandingan gambar dan analisis nilai piksel. Penggunaan `int16` dan `np.clip` membuat hasil bebas dari overflow, sehingga piksel yang sudah sangat terang tidak berubah menjadi gelap ketika ditambah, dan sebaliknya.

7. Melakukan Perbandingan Gambar

Gambar asli, gambar lebih terang, dan gambar lebih gelap ditampilkan berdampingan dalam satu baris agar perbedaannya mudah diamati.

Kode program:

```
# ==============================
# 7. MENAMPILKAN PERBANDINGAN GAMBAR
# ==============================

# Mengubah warna BGR menjadi RGB
gambar_asli_rgb = cv2.cvtColor(gambar_asli, cv2.COLOR_BGR2RGB)
gambar_terang_rgb = cv2.cvtColor(gambar_terang, cv2.COLOR_BGR2RGB)
gambar_gelap_rgb = cv2.cvtColor(gambar_gelap, cv2.COLOR_BGR2RGB)

# Membuat area untuk tiga gambar
plt.figure(figsize=(12, 5))

# Menampilkan gambar asli
plt.subplot(1, 3, 1)
plt.imshow(gambar_asli_rgb)
plt.title("Gambar Asli")
plt.axis("off")

# Menampilkan gambar lebih terang
plt.subplot(1, 3, 2)
plt.imshow(gambar_terang_rgb)
plt.title("Lebih Terang (+50)")
plt.axis("off")

# Menampilkan gambar lebih gelap
plt.subplot(1, 3, 3)
plt.imshow(gambar_gelap_rgb)
plt.title("Lebih Gelap (-50)")
plt.axis("off")

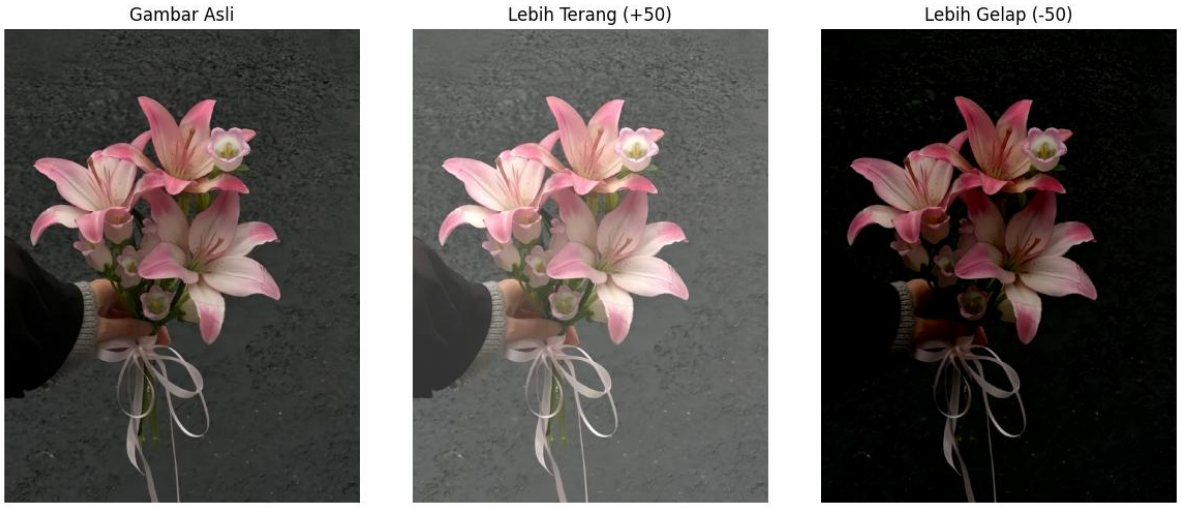
# Merapikan dan menampilkan hasil
plt.tight_layout()
plt.show()
```

✓ 0.2s

Penjelasan kode:

- `gambar_asli_rgb = cv2.cvtColor(gambar_asli, cv2.COLOR_BGR2RGB)` mengubah gambar asli dari BGR ke RGB. Dua baris berikutnya melakukan konversi yang sama pada `gambar_terang` dan `gambar_gelap`, dan hasilnya disimpan di `gambar_terang_rgb` dan `gambar_gelap_rgb`. Konversi ini dilakukan pada ketiga gambar agar warna semuanya tampil benar di Matplotlib.
- `plt.figure(figsize=(12, 5))` membuat area gambar selebar 12 inci dan setinggi 5 inci sehingga tiga gambar muat dalam satu baris.
- `plt.subplot(1, 3, 1)` membagi area menjadi 1 baris dan 3 kolom, lalu memilih posisi pertama. Pada posisi ini `plt.imshow(gambar_asli_rgb)` menampilkan gambar asli, `plt.title("Gambar Asli")` memberi judul, dan `plt.axis("off")` menyembunyikan sumbu.
- `plt.subplot(1, 3, 2)` memilih posisi kedua. `plt.imshow(gambar_terang_rgb)` menampilkan gambar yang lebih terang dengan judul `"Lebih Terang (+50)"`, dan sumbu disembunyikan dengan `plt.axis("off")`.
- `plt.subplot(1, 3, 3)` memilih posisi ketiga. `plt.imshow(gambar_gelap_rgb)` menampilkan gambar yang lebih gelap dengan judul `"Lebih Gelap (-50)"`, dan sumbu disembunyikan dengan `plt.axis("off")`.
- `plt.tight_layout()` merapikan jarak antargambar dan judul agar tidak bertumpuk, kemudian `plt.show()` menampilkan seluruh visualisasi.

Hasil eksekusi:



Gambar 2. Hasil eksekusi tahap Melakukan Perbandingan Gambar

Output berupa satu visualisasi berukuran 1200 x 500 piksel dengan tiga sumbu (`<Figure size 1200x500 with 3 Axes>`), ditunjukkan pada Gambar 2. Gambar di tengah tampak lebih

terang dari gambar asli, sedangkan gambar di kanan tampak lebih gelap. Ukuran gambar tidak berubah, hanya nilai intensitas pikselnya.

8. Menyimpan Hasil Pengolahan Citra

Gambar hasil pengolahan disimpan sebagai berkas terpisah agar dapat dibuka kembali tanpa menjalankan ulang seluruh proses.

Kode program:

```
# ==============================
# 8. MENYIMPAN HASIL PENGOLAHAN CITRA
# ==============================

# Menentukan nama file hasil
file_terang = "hasil/gambar_terang.jpg"
file_gelap = "hasil/gambar_gelap.jpg"

# Menyimpan gambar hasil Brightness Adjustment
cv2.imwrite(file_terang, gambar_terang)
cv2.imwrite(file_gelap, gambar_gelap)

print("Gambar hasil berhasil disimpan.")
print("Lokasi:", file_terang, "dan", file_gelap)
```

✓ 0.0s

Penjelasan kode:

- `file_terang = "hasil/gambar_terang.jpg"` membuat variabel berisi lokasi dan nama berkas untuk gambar terang. Bagian `hasil/` adalah nama folder tujuan dan `gambar_terang.jpg` adalah nama berkasnya.
- `file_gelap = "hasil/gambar_gelap.jpg"` melakukan hal yang sama untuk gambar gelap sehingga kedua hasil disimpan dalam berkas yang berbeda.
- `cv2.imwrite(file_terang, gambar_terang)` menulis data piksel `gambar_terang` ke berkas pada lokasi `file_terang`. Parameter pertama adalah lokasi berkas dan parameter kedua adalah data gambar. Format JPG ditentukan dari ekstensi nama berkas.
- `cv2.imwrite(file_gelap, gambar_gelap)` menyimpan `gambar_gelap` ke `file_gelap` dengan cara yang sama. Nilai kembalian `imwrite` (benar atau salah) tidak ditampung atau diperiksa dalam kode ini.
- `print("Gambar hasil berhasil disimpan.")` mencetak pesan bahwa proses penyimpanan telah dijalankan.
- `print("Lokasi:", file_terang, "dan", file_gelap)` mencetak lokasi kedua berkas. Koma memisahkan beberapa argumen, dan `print` menyambungnya dengan spasi.

Hasil eksekusi:

```
Gambar hasil berhasil disimpan.
Lokasi: hasil/gambar_terang.jpg dan hasil/gambar_gelap.jpg
```

Output menunjukkan bahwa perintah penyimpanan dijalankan dan kedua lokasi berkas tercetak. Hasilnya adalah dua berkas, yaitu `gambar_terang.jpg` dan `gambar_gelap.jpg`, di folder `hasil`, sedangkan `foto asli.jpg` tidak ditimpa. Kedua berkas inilah yang ditampilkan pada bagian C laporan.

9. Analisis Nilai Intensitas Piksel

Tahap terakhir menghitung nilai minimum, maksimum, dan rata-rata intensitas dari ketiga gambar agar perubahan kecerahan dapat dibuktikan secara numerik.

Kode program:

```python
# ==============================
# 9. ANALISIS NILAI INTENSITAS PIKSEL
# ==============================

# Menghitung statistik intensitas setiap gambar
gambar_dianalisis = [
    ("Gambar Asli", gambar_asli),
    ("Gambar Terang", gambar_terang),
    ("Gambar Gelap", gambar_gelap)
]

print("ANALISIS INTENSITAS PIXEL")
print("=" * 55)
print(f"{'Gambar':<18} | {'Minimum':>8} | {'Maksimum':>8} | {'Rata-rata':>9}")
print("-" * 55)

for nama, gambar in gambar_dianalisis:
    minimum = np.min(gambar)
    maksimum = np.max(gambar)
    rata_rata = np.mean(gambar)

    print(f"{nama:<18} | {minimum:>8} | {maksimum:>8} | {rata_rata:>9.2f}")

print("=" * 55)
```

✓ 0.0s

Penjelasan kode:

- `gambar_dianalisis = [ ... ]` membuat daftar yang berisi tiga pasangan (tuple), masing-masing terdiri dari nama gambar dan datanya: `gambar_asli`, `gambar_terang`, dan `gambar_gelap`. Dengan daftar ini, analisis dapat dilakukan dengan satu perulangan.
- `print("ANALISIS INTENSITAS PIXEL")` mencetak judul tabel, dan `print("=" * 55)` mencetak garis pemisah sepanjang 55 karakter.

- `print(f"{'Gambar':<18} | {'Minimum':>8} | {'Maksimum':>8} | {'Rata-rata':>9}")` mencetak kepala tabel. Format `:<18` meratakan teks ke kiri pada lebar 18 karakter, sedangkan `:>8` dan `:>9` meratakan ke kanan pada lebar 8 dan 9 karakter. Lalu `print("-" * 55)` mencetak garis pemisah.
- `for nama, gambar in gambar_dianalisis:` mengulang proses sebanyak tiga kali. Pada setiap putaran, pasangan dari daftar dipecah ke variabel `nama` dan `gambar`.
- `minimum = np.min(gambar)` dan `maksimum = np.max(gambar)` mencari nilai intensitas terkecil dan terbesar pada gambar yang sedang diproses. `rata_rata = np.mean(gambar)` menghitung rata-rata seluruh nilai kanal pada gambar tersebut.
- `print(f"{nama:<18} | {minimum:>8} | {maksimum:>8} | {rata_rata:>9.2f}")` mencetak satu baris data. Format `:>9.2f` meratakan ke kanan pada lebar 9 karakter dan menampilkan dua angka di belakang koma.
- `print("=" * 55)` terakhir mencetak garis penutup tabel.

Hasil eksekusi:

```
...  ANALISIS INTENSITAS PIXEL
     =======================================================
     Gambar             |  Minimum | Maksimum | Rata-rata
     -------------------------------------------------------
     Gambar Asli        |        0 |      255 |     63.58
     Gambar Terang      |       50 |      255 |    113.45
     Gambar Gelap       |        0 |      205 |     18.20
     =======================================================
```

Rata-rata intensitas gambar asli adalah 63,58. Pada gambar terang rata-rata naik menjadi 113,45, dan pada gambar gelap turun menjadi 18,20. Ini membuktikan secara numerik bahwa penambahan nilai kecerahan menaikkan intensitas piksel dan pengurangan menurunkannya. Pembahasan lengkap hasil ini disajikan pada bagian D.

C. Penjelasan Hasil Pengolahan Gambar

Gambar 3, Gambar 4, dan Gambar 5 menampilkan gambar asli serta dua hasil Brightness Adjustment dengan ukuran yang sama, yaitu 736 x 981 piksel.



Gambar 3. Gambar asli



Gambar 4. Gambar lebih terang (nilai kecerahan +50)



Gambar 5. Gambar lebih gelap (nilai kecerahan -50)

Perbedaan ketiga gambar terlihat jelas pada kecerahan keseluruhan. Pada gambar asli, latar belakang berupa permukaan abu-abu gelap, sedangkan bunga tampak sebagai bagian yang paling terang dengan warna merah muda dan putih. Pada gambar lebih terang (+50), latar belakang berubah menjadi abu-abu yang lebih terang, dan bunga juga tampak lebih cerah dan lebih pucat sehingga kontras antara bunga dan latar menjadi lebih rendah. Pada gambar lebih gelap (-50), latar belakang menjadi hampir hitam, lengan baju yang gelap hampir menyatu dengan latar, dan bunga tampak lebih redup, tetapi masih terlihat jelas karena intensitas bunga semula lebih tinggi daripada latar.

Perubahan ini terjadi karena Brightness Adjustment menambahkan atau mengurangi 50 pada intensitas setiap piksel, baik pada bunga maupun latar. Selisih intensitas antarpiksel tetap sama, sehingga bentuk dan susunan objek tidak berubah dan yang bergeser hanyalah tingkat terang gelapnya. Pembatasan pada rentang 0-255 menyebabkan piksel yang sudah mendekati 255 tidak dapat naik lebih tinggi pada gambar terang, dan piksel yang sudah mendekati 0 tidak dapat turun lebih rendah pada gambar gelap. Akibatnya, sebagian perbedaan intensitas pada area yang sangat terang atau sangat gelap berpotensi hilang.

D. Analisis Nilai Intensitas Piksel

Hasil perhitungan pada notebook dirangkum pada Tabel 1.

Tabel 1. Nilai intensitas piksel pada setiap gambar

Jenis Gambar	Minimum	Maksimum	Rata-rata
Gambar Asli	0	255	63,58
Gambar Terang	50	255	113,45
Gambar Gelap	0	205	18,20

Nilai minimum adalah intensitas terendah yang ditemukan pada gambar, yaitu piksel yang paling gelap. Nilai maksimum adalah intensitas tertinggi atau piksel yang paling terang. Nilai rata-rata menggambarkan tingkat kecerahan gambar secara keseluruhan karena merupakan hasil penjumlahan seluruh nilai intensitas dibagi jumlah nilainya. Ketiga nilai ini melengkapi pengamatan visual dengan angka yang dapat dibandingkan.

Rata-rata intensitas meningkat dari 63,58 pada gambar asli menjadi 113,45 pada gambar terang karena setiap piksel ditambah 50. Sebaliknya, rata-rata menurun menjadi 18,20 pada gambar gelap karena setiap piksel dikurangi 50. Kenaikan rata-rata sebesar 49,87 (113,45 - 63,58) hampir sama dengan nilai +50, sedangkan penurunannya sebesar 45,38 (63,58 - 18,20) lebih kecil dari 50. Selisih ini terjadi karena adanya pembatasan intensitas pada rentang 0-255, sehingga tidak semua piksel mengalami perubahan penuh sebesar 50.

Pembatasan tersebut juga terlihat pada nilai minimum dan maksimum. Pada gambar terang, nilai minimum naik dari 0 menjadi 50 karena piksel paling gelap ditambah 50, tetapi nilai maksimum tetap 255 karena hasil penjumlahan yang melebihi 255 dipotong menjadi 255. Pada gambar gelap, nilai maksimum turun dari 255 menjadi 205 (255 - 50), tetapi nilai minimum tetap 0 karena hasil pengurangan yang kurang dari 0 dipotong menjadi 0. Dengan demikian, nilai minimum atau maksimum tidak selalu berubah penuh sebesar 50, tergantung pada batas rentang piksel.

Angka pada Tabel 1 diambil dari output notebook yang dihitung langsung dari data gambar di memori. Karena format JPG bersifat lossy, nilai yang dihitung ulang dari berkas JPG hasil simpan dapat berbeda sedikit dari angka tersebut.

E. Kesimpulan

Berdasarkan program yang telah dibuat, metode Brightness Adjustment berhasil diterapkan untuk mengubah kecerahan citra digital. Dengan menambahkan nilai +50 dan -50 pada seluruh piksel, program menghasilkan gambar yang lebih terang dan gambar yang lebih gelap dari gambar asli tanpa mengubah ukuran maupun bentuk objek. Penggunaan tipe data `int16` dan fungsi `np.clip` menjaga hasil perhitungan tetap berada pada rentang 0-255.

Secara visual, gambar terang memiliki latar dan bunga yang lebih cerah, sedangkan gambar gelap memiliki latar yang hampir hitam dengan bunga yang lebih redup. Secara numerik, rata-rata intensitas piksel naik dari 63,58 menjadi 113,45 pada gambar terang dan turun menjadi 18,20 pada gambar gelap. Nilai minimum dan maksimum yang tidak berubah penuh menunjukkan pengaruh pembatasan intensitas pada rentang 0-255. Hasil visual dan numerik tersebut membuktikan bahwa Brightness Adjustment dapat menaikkan maupun menurunkan kecerahan citra sesuai nilai perubahan yang diberikan.